



ITChat

Detecta. Protege. Conversa seguro.

Mensajería en tiempo real con clasificación automática de amenazas mediante Machine Learning, desplegada sobre infraestructura de red y nube propia.

Joseph Shakalo

A01784107

Gabriel Edid

A01782146

Marcos Dayan

A01782876

¿Qué es ITChat?

Aplicación de mensajería directa (1 a 1) en tiempo real. Cada mensaje enviado se clasifica al instante con un modelo de Machine Learning. La etiqueta persiste en la base de datos y el destinatario la recibe vía WebSocket junto con el mensaje.



ham

Mensaje legítimo: conversación normal entre usuarios.



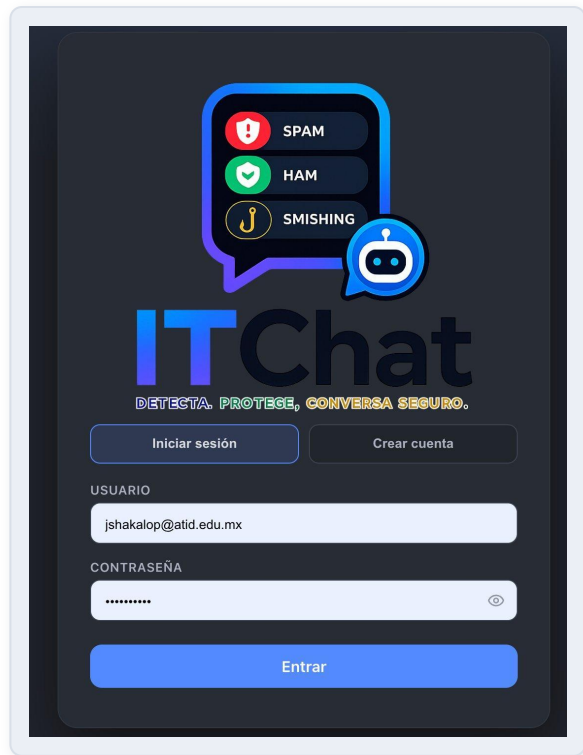
spam

Publicidad o mensaje no deseado: promociones, ofertas masivas.



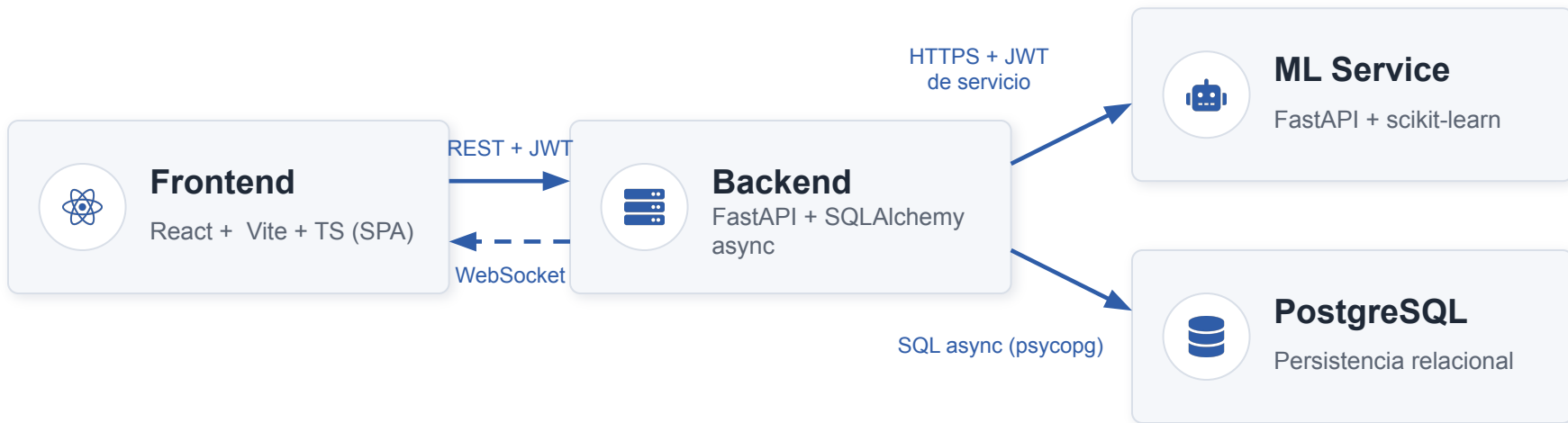
smishing

Phishing por SMS: intento de robo de datos o credenciales.



Pantalla de inicio de sesión

Cuatro servicios desacoplados



Un solo repo con dependencias independientes por servicio (uv / npm)



Comunicación solo por HTTPS y WebSocket: sin acoplamiento



Cada servicio se despliega en su propia instancia de la Nube.

SPA en React + TypeScript + Vite



Páginas

Login/registro (con bio y avatar), Dashboard con sidebar de conversaciones, búsqueda de usuarios y ChatWindow.



Sesion persistente

JWT y datos del usuario en localStorage: la sesión sobrevive recargas.



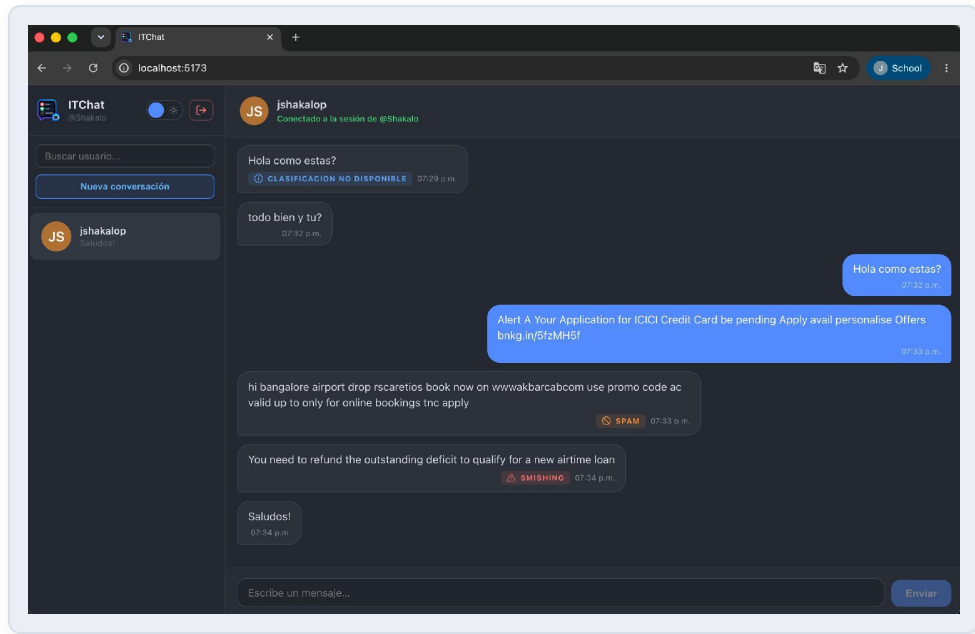
Tiempo real

Un único WebSocket por usuario autenticado que reacciona a eventos de mensajes creados.



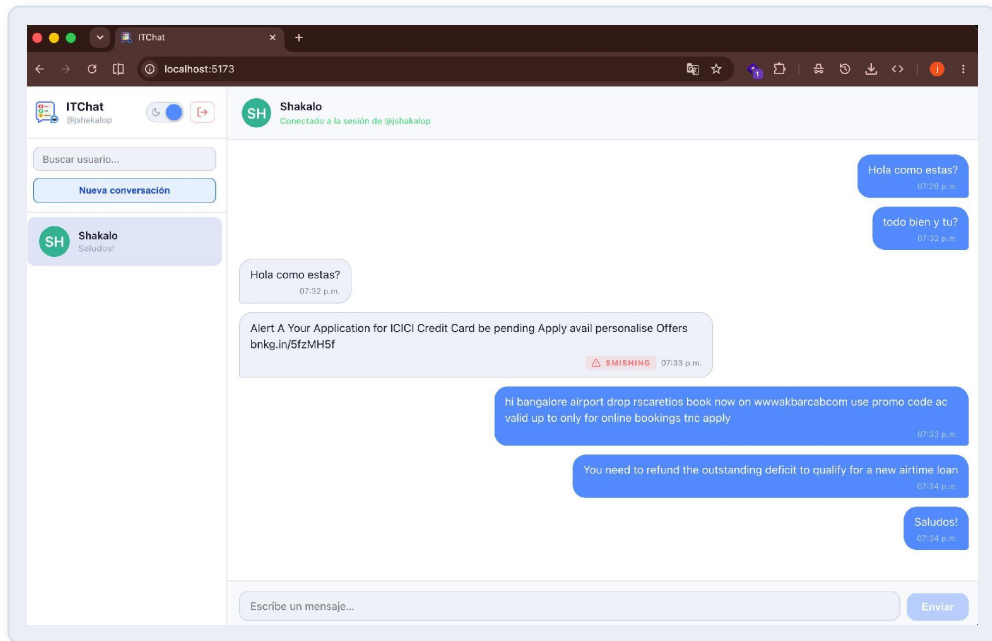
Capa de API

Centraliza el fetch REST y la URL del WebSocket vía.

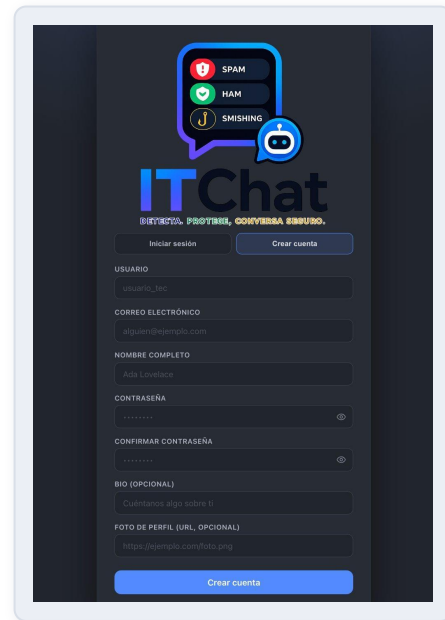


Dashboard: cada mensaje llega con su etiqueta de clasificación en tiempo real

UI/UX, accesibilidad y usabilidad



Modo claro con toggle persistente



Registro con validación



Tema claro / oscuro persistente



Badges con color + icono + texto



Estados visibles ante fallos del ML



Mostrar / ocultar contraseña

API REST con FastAPI (async)

Método	Ruta	Auth	Descripción
POST	/v1/register · /v1/login	No	Crear cuenta / iniciar sesión, devuelven JWT
GET	/v1/users/search?query=	JWT	Buscar usuarios por nombre
GET	/v1/conversations	JWT	Listar chats del usuario
POST	/v1/conversations/direct/{id}	JWT	Crear o recuperar chat directo
GET	/v1/direct_messages/{id}	JWT	Historial de mensajes de un chat
POST	/v1/messages	JWT	Enviar mensaje → clasifica con el ML service
WS	/v1/ws?token=	JWT	Canal push en tiempo real (message_created)
GET	/health	No	Health check (lo consume el load balancer)



Stack

FastAPI + SQLAlchemy 2.0 async + Uvicorn; driver psycopg (Postgres) y aiosqlite (tests).



Servicio systemd

itchat-backend.service con Restart=always: si el proceso cae, se reinicia solo en 5 s.



Auto-bootstrap

Al arrancar crea la BD itchat y sus tablas si no existen: cero pasos manuales.

Tiempo real con WebSockets

1

Conexión

El frontend abre una sesión con el ws al iniciar sesión.
Un solo socket por usuario, no por chat.

2

Validación

El backend valida el JWT del query param y registra la conexión.

3

Push

Al guardarse un mensaje, el backend emite la creación del mensaje solo al destinatario conectado.

Evento recibido por el destinatario

```
{
  "type": "message_created",
  "conversation_id": 1,
  "message": {
    "sender_id": 2,
    "content": "Verifica tu cuenta...",
    "classification_label": "smishing",
    "created_at": "2026-06-08T18:00:00Z"
  },
  "sender": { "username": "otro_usuario" }
}
```

Base de datos relacional (PostgreSQL)

Tabla	Columnas clave	Relación
users	id, username, email, password_hash, full_name, bio, avatar_url	—
conversations	id, created_at	1:N con messages y participants
conversation_participants	conversation_id, user_id (UNIQUE por par)	Puente N:M usuarios ↔ conversaciones
messages	conversation_id, sender_id, content, classification_label	N:1 con conversación y remitente



Normalización (3FN)

Sin grupos repetidos ni dependencias parciales o transitivas: los participantes viven en su tabla puente, no en una columna-lista.



Históricos

Todo mensaje queda persistido con su classification_label y created_at: el historial conserva la evidencia de amenazas.



Producción vs tests

PostgreSQL (psycopg async) en producción; SQLite en memoria (aiosqlite) en tests.

Defensa en capas



Contraseñas

PBKDF2-HMAC-SHA256 con 600,000 iteraciones + salt aleatorio por usuario. Nunca se guarda texto plano.



JWT de usuario

Login/registro devuelven token firmado (HS256, expira en 60 min). Endpoints protegidos necesitan Authorization: Bearer.



JWT service-to-service

Backend ↔ ML service usan un segundo sistema JWT con secreto, issuer, audience y subject propios, y expiración de 60 segundos.



Base de datos

pg_hba.conf solo acepta conexiones del CIDR de los backends, con autenticación scram-sha-256.



Red

Hacia la red del Tec solo se exponen los puertos 445 y 22; SSH únicamente a través del jump server.



CORS

CORS_ALLOW_ORIGINS limita el origen del frontend permitido en producción.

Dataset: tres fuentes reales reconciliadas

Mishra & Soni (2022)

Mendeley · CC BY 4.0

Fuente principal: ya trae las 3 clases (ham, spam, smishing).

UCI SMS Spam Collection

Benchmark clásico

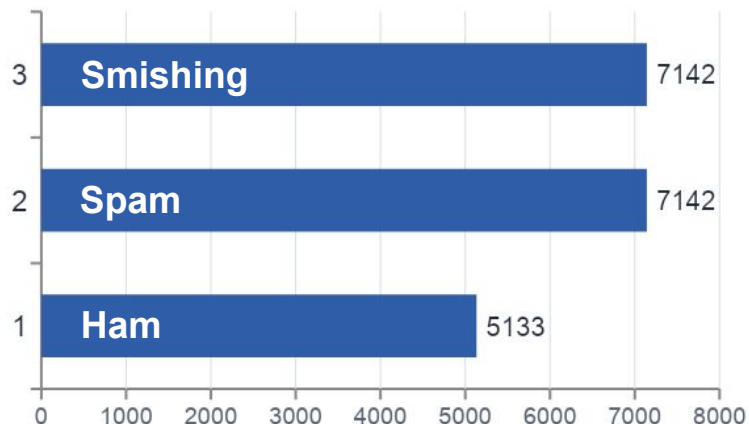
Refuerza ham y spam.

Combined Smishing Dataset

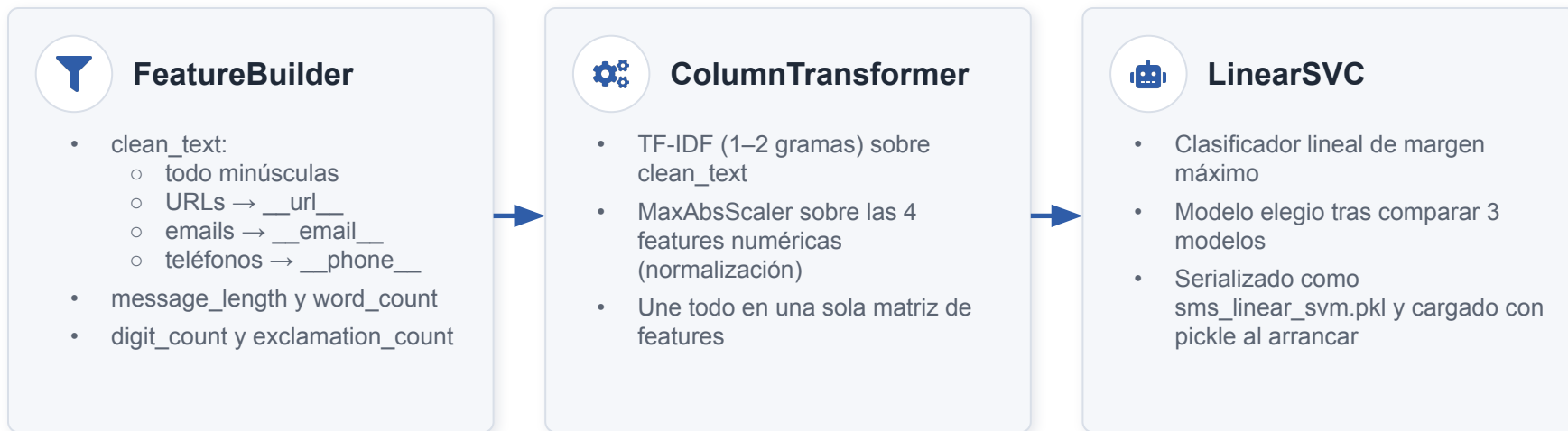
Hosseinpour et al.

Consolida varias fuentes públicas; refuerza smishing y spam.

19,417 mensajes SMS consolidados

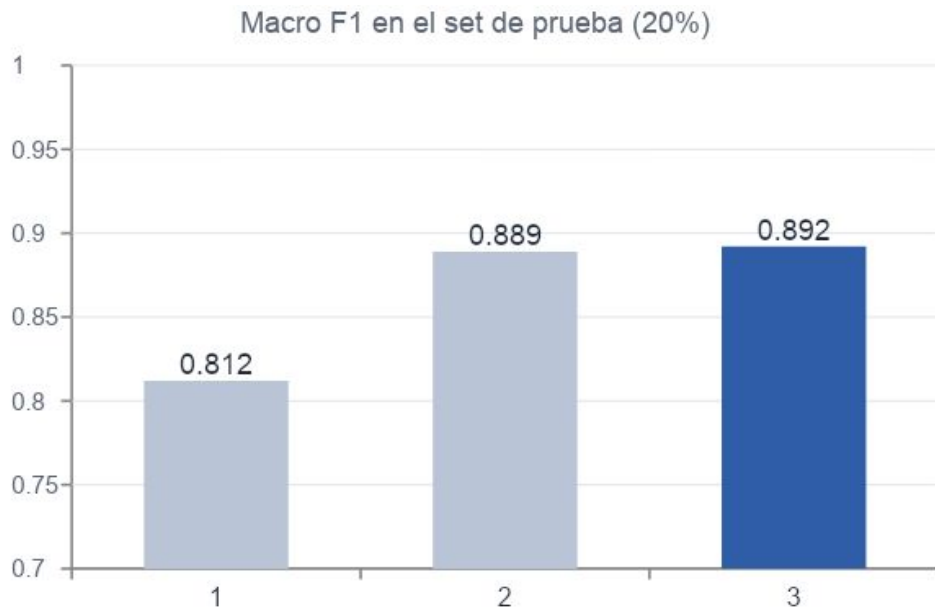


Pipeline de clasificación (scikit-learn)



Mismo código en entrenamiento y producción: el FeatureBuilder es un transformer custom dentro del pipeline, así que el `.pkl` aplica exactamente el mismo preprocesamiento que se usó al entrenar, sin divergencia entre notebook y servicio.

Comparación de modelos y resultados



0.886

accuracy del Linear SVM

0.892

macro F1 del Linear SVM

0.94

F1 de la clase ham

El modelo como microservicio aislado

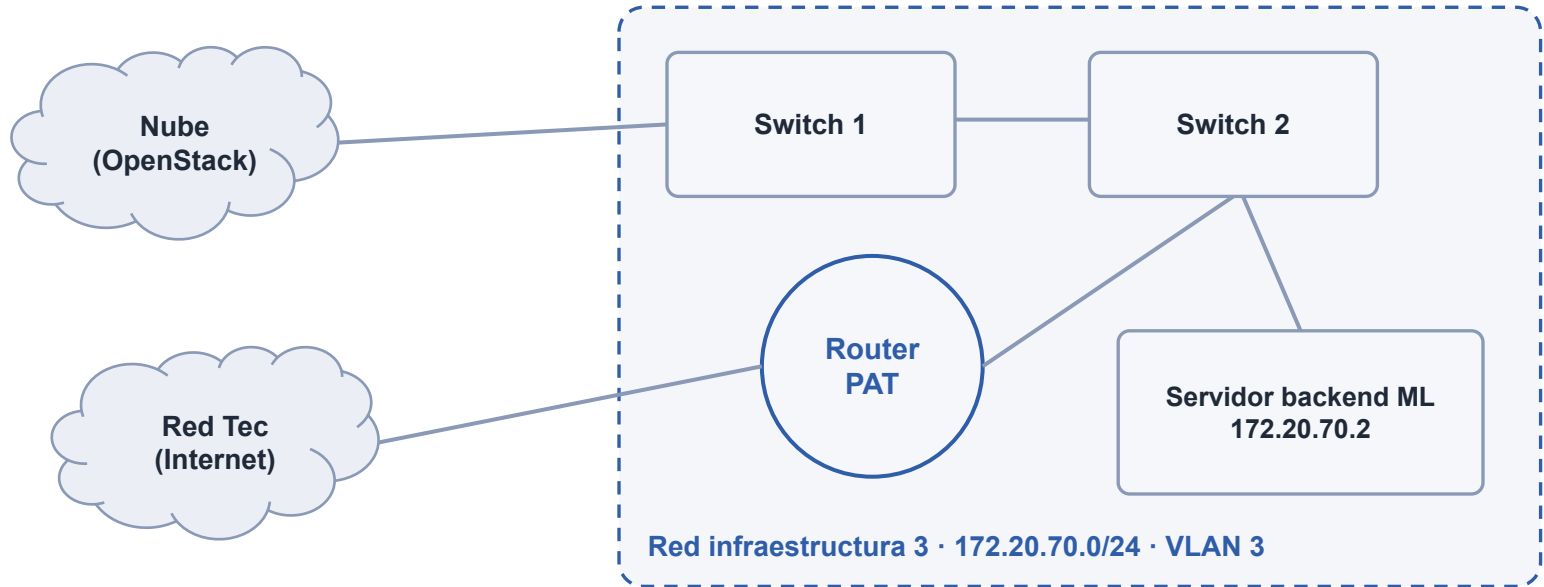


Vida de un mensaje, end-to-end



Latencia percibida mínima: la clasificación ocurre dentro del mismo request de envío y el destinatario recibe el mensaje ya etiquetado, sin estados intermedios visibles.

Topología física de la red



- PAT en el router: reparte la salida a internet a toda la subred privada
- Hacia la red del Tec solo se exponen los puertos 443 (HTTPS) y 22 (SSH)
- La VLAN conecta las premisas físicas con la nube OpenStack del laboratorio

Despliegue en OpenStack: 5 instancias



Diseño escalable, seguro y redundante: el LB permite agregar más backends sin tocar el frontend, los security groups restringen cada capa (SSH solo desde dentro de la red, Postgres solo desde los backends) y la caída de un backend no tira el servicio.

Infraestructura como código con cloud-init



`gjm-front-cloud-init`

Instala Node 20 + nginx, clona el repo, hace npm ci && npm run build y publica dist/ en /var/www/html.



`gjm-back{1,2}-cloud-init`

Instala uv, clona el repo, escribe /etc/itchat/backend.env y registra itchat-backend.service en systemd.



`gjm-db-cloud-init`

Instala PostgreSQL, habilita listen_addresses='*' y restringe pg_hba al CIDR de backends con scram-sha-256.



`gjm-lb-cloud-init + itchat.conf`

Despliega el nginx balanceador con el upstream de los dos backends.



Reproducible

Levantar una instancia nueva = pegar su cloud-init.
Cero configuración manual.



Secretos como plantilla

Placeholders (`__JWT_SECRET_KEY__`, `__DB_PRIVATE_IP__`) se inyectan al desplegar:
nada sensible en el repo.



Auto-recuperación

systemd reinicia el backend ante cualquier caída (RestartSec=5).

Load balancer nginx: escalable y redundante

`/etc/nginx/conf.d/itchat.conf`

```
upstream itchat_backend {
    least_conn;
    server back1:8000 max_fails=3
                fail_timeout=30s;
    server back2:8000 max_fails=3
                fail_timeout=30s;
    keepalive 32;
}
location /v1/ws {
    proxy_set_header Upgrade ...;
    proxy_read_timeout 3600s;
}
```



least_conn

Cada request va al backend con menos conexiones activas: mejor que round-robin con WebSockets de larga vida.



Health checks pasivos

Tras 3 fallos, el backend se saca de rotación 30 s. /health sin auth permite sondear el estado.



WebSockets de larga vida

Se hace upgrade de protocolo en /v1/ws y proxy_read_timeout de 1h para no cortar sockets activos.



keepalive 32

Reutiliza conexiones LB→backend, reduciendo latencia por handshake.

Pruebas realizadas



Backend · 17 tests (pytest)

- Corren con SQLite en memoria y un stub del modelo: no requieren Postgres ni el .pkl
- Cubren auth, conversaciones, mensajes y el contrato con el ML service
- uv run pytest en CI o local, en segundos



Modelo · Evaluación formal

- Split 80/20 estratificado por clase
- Classification report y matrices de confusión por modelo
- Pruebas con mensajes nuevos fuera del dataset (sanity check)



Infraestructura · End-to-end

- Health checks del LB contra /health de cada backend
- Conectividad SSH por la cadena: laptop → PAT → jump → instancias
- Flujo completo verificado: login, chat y clasificación en vivo

Conclusiones y trabajo futuro



Sistema completo y funcional

De la UI al fierro: SPA, API, ML, base de datos, VLAN propia y despliegue automatizado en OpenStack.



ML honesto y explicable

Linear SVM con macro F1 de 0.892 sobre 19,417 mensajes reales, los errores restantes están entre spam y smishing, las clases más parecidas.



Seguridad en cada capa

PBKDF2, doble JWT, scram-sha-256, security groups y un único punto de entrada SSH.



Listo para crecer

Backends redundantes tras un LB y aprovisionamiento por cloud-init: escalar es agregar instancias.

Trabajo futuro

- Migraciones formales de BD (Alembic)
- Distinguir mejor spam vs smishing (subtipos, embeddings)
- Creación de chats en grupo
- Modelo de reconocimiento de imágenes sensibles
- Reentrenamiento periódico con mensajes reportados
- Levantar una instancia más de Backend para el Load Balancer (falta de instancias)

¡Gracias!